COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/7/26	**DATE SUBMITTED:**	10/7/26
NAME:	Pangyarihan,Jenny Rose Peñaflorida,Wrio Carlo	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 11.1 BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

ILOA. Measure the runtime of algorithms using theoretical analysis
Provide an analysis of the graph for the worse case of the algorithm.

Analysis:

In the worst case, none of the values in array y appear in array x, so diff never returns early. The loop runs n times, and each call to search scans all m elements of x. With m = n, the total is n × n = n^2 comparisons, or n^2+ n if the loop overhead is counted, as in the graph.

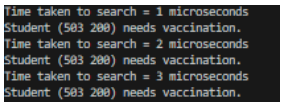
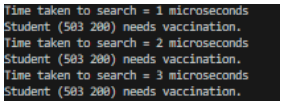
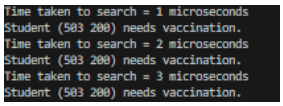
The graph confirms this. The curve bends upward instead of forming a straight line, and the increases get larger as n grows (310 from n = 10 to 20, then 510 from n = 20 to 30). Doubling n from 10 to 20 raises the operations from 110 to 420, roughly four times as many, which is the behavior of a quadratic function. At n = 30 the n^2 term already accounts for about 97% of the total, so the +n term becomes negligible as the input grows.

Therefore, the worst-case running time of diff is $O(n^2)$. This is acceptable for small arrays but gets slow quickly for large ones, since every doubling of the input size quadruples the work. A faster approach would be to sort one array and use binary search for each lookup, which brings the cost down to O(n log n

ILOB: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Provide your own analysis of the outcome listed in this table.

Peñaflorida

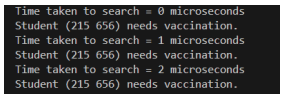
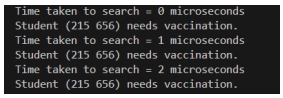
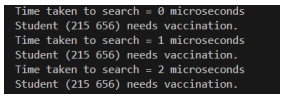
Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	1 micro seconds		Binary search only needs 10 halvings so it instantly finishes
10,000	2 micro seconds		10 times bigger but the speed only rose to 2 micro seconds
100,000	3 micro seconds		100 times bigger but only rose to 3 micro seconds

Analysis:

This matches the theoretical analysis. Binary search discards half of the remaining range at every step, so the number of steps is about log^2(N). That is roughly 10 steps for 1,000 students, 13 for 10,000 and 17 for 100,000. A 100× larger list adds only about 7 steps, which is why the measured time barely changed. A linear search would have needed up to 100,000 comparisons for the largest list, so its time would have grown about 100×. The observed trend is therefore logarithmic, O(log n), and not linear.

Small differences between the measured values and the theory are expected. The times are only a few microseconds, so CPU load, caching and clock resolution affect the numbers, and running the program again may give slightly different results. Only the search was timed, not the sorting, which is O(n log n) and would take much longer than the search itself. The student (503 200) needed vaccination in every run because the student was either not on the list or had not been vaccinated.The experimental results agree with the theoretical analysis. Binary search scales very well, which makes it a good choice for large sorted lists like the flu shot records, where the program must be fast and efficient

Pangyarihan:

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 micro seconds		Search finished in 0, too fast for the timer to register.
10,000	1 micro seconds		Search took 1, time went up slightly as the input.
100,000	2 micro seconds		Search took 2, time kept rising but only by a tiny amount even though the is 100x larger than the first run.

Analysis:

When the number of records increased from 1,000 to 100,000, the search time rose only from 0 to 2 microseconds. This shows that the search stays quick even with much larger data. A slower method that checks each item one at a time would take far longer as the data grows. Because of this, the program appears to handle big inputs efficiently. Since the measured times are so tiny, they may not be perfectly precise, but the overall trend shows that the search is efficient.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis
For each of the problems below, provide the following:
Theoretical Analysis
Experimental Analysis
Analysis and Comparison
Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
for i = 0 to n-1 do
for j = i+1 to n-1 do
if A[i] equals to A[j] then
return false
return true
```

```
--- Problem 1: Unique ---
n = 1000 result = 1 time = 13339 microseconds
n = 2000 result = 1 time = 53069 microseconds
n = 4000 result = 1 time = 105037 microseconds
n = 8000 result = 1 time = 610137 microseconds
```

PANGYARIHAN:

Theoretical analysis: Unique checks every element against the others using nested loops, which comes to roughly n(n-1)/2 comparisons. This makes it O(n²), so each time n doubles, the runtime should grow about fourfold.

Experimental analysis:
The recorded times were 13,339 µs at n = 1000, 53,069 µs at n = 2000, 105,037 µs at n = 4000, and 610,137 µs at n = 8000. Going from 1000 to 2000, the runtime increased roughly 4×, in line with the quadratic prediction. The next step, from 2000 to 4000, only about doubled the time, while the jump from 4000 to 8000 was around 5.8×.

Analysis and comparison:

Looking at the whole range, n became 8 times larger and the runtime grew by about 46 times, which is reasonably near the expected factor of 64. The uneven changes between steps are probably caused by timing noise from CPU load, caching, and clock resolution. Averaging multiple runs would produce a more reliable picture.

```
--- Problem 2: Power ---
n = 10 | rpower = 0 microseconds | brpower = 0 microseconds | results 1.001 / 1.001
n = 100 | rpower = 0 microseconds | brpower = 0 microseconds | results 1.01005 / 1.01005
n = 1000 | rpower = 43 microseconds | brpower = 0 microseconds | results 1.10517 / 1.10517
n = 10000 | rpower = 431 microseconds | brpower = 0 microseconds | results 2.71815 / 2.71815
```

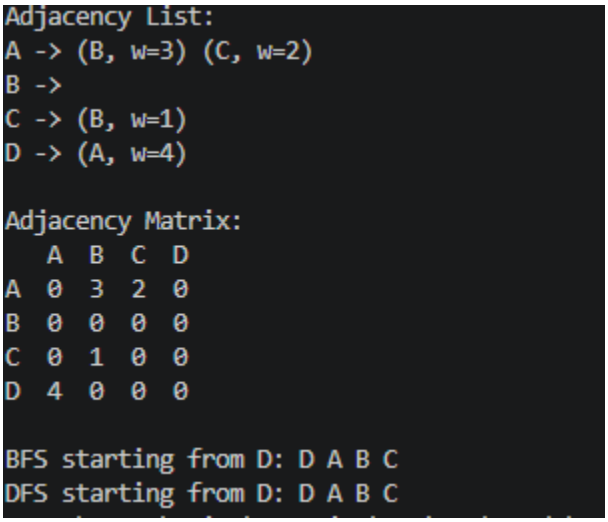
PANGYARIHAN:

Theoretical analysis: The recursive rpower makes one call for every decrease in the exponent, giving it O(n) time and O(n) stack depth. The binary version, brpower, cuts the exponent in half at each step, so it runs in O(log n).

Experimental analysis: Both functions returned the same values (1.001, 1.01005, 1.10517, 2.71815), confirming that each is correct. At n = 10 and n = 100, both registered 0 µs. At n = 1000, rpower needed 43 µs, and at n = 10,000 it needed 431 µs, whereas brpower remained at 0 µs in every case.

Analysis and comparison: When n grew tenfold, rpower's runtime also grew about tenfold, which supports linear behavior. Even at n = 10,000, brpower needs only about 14 multiplications, which is too quick for a microsecond timer to detect, matching its O(log n) complexity. The 0 µs readings at small n reflect the timer's limited resolution, not an absence of work. In short, the results show that the binary method scales much better than the linear recursive one.

Supplementary Output Peñaflorida



Theoretical Analysis/Experimental analysis/Analysis and comparison:

Worst case (all elements unique, so there is no early return):
The inner loop runs (n−1) + (n−2) + … + 1 times.
Total comparisons = n(n−1)/2 = (n² − n)/2, which is O(n²).
Best case: A[0] equals A[1], so only 1 comparison is needed, which is O(1).

Theory says the worst case needs about n²/2 comparisons, so doubling n should make the work about 4× larger. The experiment matches: 2,720 → 11,085 is 4.08×, 11,085 → 44,151 is 3.98×, and 44,151 → 171,211 is 3.88×. The small differences from exactly 4× come from CPU load, caching and clock resolution. Both analyses agree that Unique runs in O(n²) time, so it becomes slow for large arrays. A faster approach is to sort the array and compare neighbors (O(n log n)) or to use a hash set (O(n) on average).

rpower: each call reduces n by 1, so T(n) = T(n−1) + c, which gives T(n) = cn + d, or O(n). It makes n + 1 calls and n multiplications.
brpower: each call halves n, so T(n) = T(n/2) + c, which gives T(n) ≈ c·log₂n + d, or O(log n). For n = 10,000 it needs only about 14 calls.The experiment agrees with the theory. For rpower, n grew 10× (1,000 to 10,000) and the time grew from 35 to 344 microseconds, about 10×, which is linear growth. For brpower, the time stayed at 0 microseconds for every n. That doesn't mean it took no time. It finished in under 1 microsecond, too fast for the clock to register, because it needs only about 14 recursive calls even at n = 10,000. For small n (10 and 100), both functions are so fast that the difference is invisible. Both returned the same results (1.001, 1.01005, 1.10517, 2.71815), so they are equally correct, but brpower is far more efficient. The O(log n) divide-and-conquer approach scales much better than the O(n) approach, and the gap widens as n grows.

CONCLUSION:

Peñaflorida

In this activity, I realized that an algorithm's real efficiency comes down to how its runtime scales with input size n, not just how fast a specific computer runs it. Combining theoretical Big-O analysis with empirical benchmarking using std::chrono showed how theory plays out in practice. In ILO A, the diff function's nested logic clearly demonstrated quadratic $O(n^2)$ growth. On the other hand, ILO B showed the power of binary search—its runtime barely budged as N grew from $1,000$ to $100,000$, perfectly matching $O(\log n)$ behavior. These tests made it clear that while Big-O worst-case bounds give us the expected baseline, real-world execution always comes with small variations due to CPU load and system clock limitations.

The supplementary problems backed this up even further. In Problem 1, the Unique algorithm's $O(n^2)$ complexity was obvious from the timing data: every time n doubled, the runtime quadrupled, jumping from $2,720\ \mu\text{s}$ all the way to $171,211\ \mu\text{s}$. Problem 2 gave a sharp comparison between linear and logarithmic strategies. As n went up tenfold from $1,000$ to $10,000$, rpower ($O(n)$) took about $10\times$ longer, while brpower ($O(\log n)$) finished because it only needed about 14 recursive calls. Even though both functions gave the exact same results, brpower was infinitely more practical for large inputs.

Overall, this assignment proved that choosing smart algorithmic strategies like divide-and-conquer does far more for performance than relying on faster hardware. Going forward, I want to work on spotting Big-O complexities from source code faster, practice solving recurrence relations like $T(n) = T(n/2) + c$ on paper, and refine my benchmarking habits by averaging multiple test runs to smooth out background timing noise.

Pangyarihan:

This activity showed me that an algorithm's efficiency depends on how its runtime grows as the input gets larger, not on how quickly one particular machine runs it. By combining Big-O analysis with actual timing using std::chrono, I saw theory confirmed in practice: the diff function in ILO A slowed down quadratically, while binary search in ILO B stayed almost flat even as N rose from 1,000 to 100,000, consistent with O(log n). The supplementary problems reinforced this, since the Unique algorithm's runtime roughly quadrupled each time n doubled, matching its O(n²) complexity. I also learned that real measurements always wobble a bit because of CPU load and clock limitations, so worst-case bounds are best treated as a baseline rather than an exact prediction.

Overall, I think I did well because my results matched the expected complexities, though I can improve by running more trials and averaging them to cut down timing noise, and by getting quicker at spotting an algorithm's complexity straight from its code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.